Simultaneous Multi-Way Alignment of LoRA Ensembles via Generalized Procrustes Analysis

Asveth Vijayakumar

Contents

1 Problem Description

1.1 Representation Collapse in Parameter-Efficient Fine-Tuning

The prevalent pre-train-then-finetune paradigm often leads to a degradation of a model's general purpose knowledge, resulting in a decline in out-of-distribution (OOD) performance. While multi task model merging can create a single model capable of solving all tasks without retraining, this success does not transfer well when merging Low-Rank Adaptation (LoRA) models.

Empirical evidence reveals that independent LoRA models exhibit catastrophic interference and representation collapse when combined naively. This occurs because the low-rank constraints force models to pick specific, often misaligned, subspaces to extract information for their assigned tasks, leading to low activation alignment.

1.2 The Orthogonality Paradox and SVD Alignment

Recent literature has demonstrated that the parameters of LoRA models showcase a significantly lower degree of functional alignment compared to their fully-finetuned counterparts. Frameworks such as KnOTS [6] resolve this by utilizing Singular Value Decomposition (SVD) to jointly transform the task-updates of different LoRA models into a shared, aligned orthogonal space, where standard merging methods can be safely applied.

1.3 The Multi-Way Alignment Bottleneck

Current state-of-the-art alignment techniques rely heavily on concatenation to build joint task spaces. However, concatenating the matrices of N different models scales poorly. This creates an $\mathcal{O}(N)$ memory bottleneck *in the alignment phase* that makes simultaneous multi-task merging computationally prohibitive on constrained hardware, forcing researchers to rely on suboptimal pairwise alignment methods. Recently, Achara et al. [1] addressed a similar limitation in the context of the Platonic Representation Hypothesis by demonstrating that Generalized Procrustes Analysis (GPA) can be successfully adapted for mapping latent spaces across multiple neural representations simultaneously, bypassing the need for pairwise scaling.

1.4 Theoretical Justification: The Geometry of Procrustes Analysis

To understand why simultaneous alignment is highly effective, it is useful to view the weights of neural networks through a geometric lens. In the context of LoRA, an adapter modifies a frozen base model layer using two low-rank matrices: $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$. When independent models are fine-tuned on distinct tasks, gradient descent forces each model to learn its task-specific features in a differently "rotated" r-dimensional subspace. Naively averaging these matrices is equivalent to averaging misaligned coordinate systems, resulting in representation collapse.

Generalized Procrustes Analysis (GPA), originally introduced by Gower [2], is a statistical shape analysis method designed to optimally superimpose multiple geometric configurations. By treating the column spaces of the A_i matrices as distinct geometric "shapes," GPA iteratively translates and rotates them until they are optimally aligned to a shared, latent consensus matrix C. This approach yields two critical advantages:

- **Geometric Feature Alignment:** By mathematically rotating all models into the exact same coordinate system *before* merging, we ensure that homologous features are aligned. This prevents the destructive interference that washes out ensemble knowledge during direct parameter addition.
- **Alignment-Phase Memory Efficiency:** Current methods concatenate the full updates of all models into one massive joint matrix, scaling alignment-phase memory requirements linearly [6]. Our GPA approach operates strictly on the low-rank matrices ($r \times d_{in}$) iteratively. The size of the matrices undergoing SVD remains constant, achieving $\mathcal{O}(1)$ *alignment-phase* memory scaling. Note that adapter training and model inference retain the same VRAM requirements as any standard QLoRA workflow; the $\mathcal{O}(1)$ property applies specifically to the alignment computation.

1.5 Project Aims

This project aims to resolve the alignment-phase memory bottleneck of multi-task LoRA merging by introducing a novel application of Generalized Procrustes Analysis (GPA) to the parameter-efficient subspace.

Our core mathematical objective minimizes the Frobenius distance across all models:

$$\min_{Q_1,\ldots,Q_N,C} \sum_{i=1}^{N} \|Q_i A_i - C\|_F^2 \tag{1}$$

Subject to the strict orthogonality constraint:

$$Q_i Q_i^T = I_r \quad \forall i \tag{2}$$

Because the inverse of an orthogonal matrix is its transpose, applying Q_i to A_i and Q_i^T to B_i preserves the functional output of the adapter perfectly while unifying their geometric coordinate systems:

$$\Delta W_i = B_i A_i = B_i (Q_i^T Q_i) A_i = (B_i Q_i^T)(Q_i A_i) \tag{3}$$

By strictly operating in the low-rank space (e.g., $r = 16$), we hypothesize this algorithm will achieve state-of-the-art multi-model alignment with an $\mathcal{O}(1)$ alignment-phase memory footprint, eliminating the need for massive joint-matrix concatenations.

1.6 Post-Alignment Merging Strategy

The GPA procedure described above solves the *alignment* phase: it produces orthogonal rotations Q_i and a consensus C such that all $Q_i A_i$ share a common coordinate system. However, alignment alone does not specify how to produce the final merged model. This section describes the complete merging pipeline.

1.6.1 Why Consensus Replacement is Insufficient

A naïve approach would replace every aligned $\tilde{A}_i = Q_i A_i$ with the consensus matrix C and then average the corresponding $\tilde{B}_i = B_i Q_i^T$ matrices. This discards all task-specific information encoded in the A factors and assumes that the adapters differ only in their B projections, which is an assumption that is unlikely to hold when adapters are trained on substantially different tasks. We include this as an **ablation** but not as the primary method.

1.6.2 Why Reconstruction Defeats Alignment

An alternative approach would reconstruct the full task updates $\Delta\tilde{W}_i = \tilde{B}_i \tilde{A}_i$ from the aligned factors and then apply conflict-resolution methods such as TIES-Merging to the set $\{\Delta\tilde{W}_i\}_{i=1}^N$. However, the functional invariance property (Equation 3) guarantees that $\tilde{B}_i \tilde{A}_i = (B_i Q_i^T)(Q_i A_i) = B_i A_i = \Delta W_i$. That is, the reconstructed matrices are *mathematically identical* to the original, unaligned task updates. Any merging method applied to $\{\Delta\tilde{W}_i\}$ therefore operates on the same inputs it would have received without GPA alignment, rendering the alignment step a no-op with respect to the final merged result. Crucially, this pitfall also forfeits the memory advantage of our approach, since the reconstructed $\Delta\tilde{W}_i \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ are full-rank matrices.

1.6.3 Primary Method: TIES-Merging in the Aligned Factor Space

The key insight, shared with KnOTS [6], is that merging must occur *within* the aligned coordinate system, before any reconstruction that would allow the rotations to cancel. KnOTS achieves this by decomposing concatenated task updates via SVD into a shared component $U\Sigma$ and task-specific components $[V^{(i)}]^T$, then applying merging methods directly to the $V^{(i)}$ matrices while holding $U\Sigma$ fixed.

Our GPA framework produces a structurally analogous decomposition. After alignment, each task update is expressed as $\Delta W_i = \tilde{B}_i \cdot \tilde{A}_i$, where:

- $\tilde{A}_i = Q_i A_i \in \mathbb{R}^{r \times d_{\text{in}}}$ are the **input-side** projections, all aligned to the common coordinate system defined by GPA (analogous to KnOTS's $[V^{(i)}]^T$);

- $\tilde{B}_i = B_i Q_i^T \in \mathbb{R}^{d_{\text{out}} \times r}$ are the **output-side** projections. Unlike KnOTS's $U\Sigma$, which is a single shared matrix by construction, the $\tilde{B}_i$ remain task-specific; we resolve this by averaging them (Step 4).

Since GPA explicitly minimises $\sum_{i=1}^{N} \|Q_i A_i - C\|_F^2$, the aligned $\tilde{A}_i$ matrices are optimised to lie close together, making element-wise conflict resolution meaningful: position (j, k) in $\tilde{A}_1$ approximately corresponds to the same feature direction as position (j, k) in $\tilde{A}_2$, with the quality of this correspondence governed by the GPA residual (measured directly in Experiment 1 of Section 3). Our primary merging strategy therefore applies TIES-Merging to the aligned $\tilde{A}_i$ matrices:

1. **Align:** Run GPA to obtain orthogonal rotations Q_i and consensus C.
2. **Transform:** Compute aligned factor pairs: $\tilde{A}_i = Q_i A_i$ and $\tilde{B}_i = B_i Q_i^T$ for each adapter i.
3. **Merge $\tilde{A}$ via TIES:** Apply TIES-Merging [7] to $\{\tilde{A}_i\}_{i=1}^{N}$:
   (a) **Trim:** Retain only the top-k% parameters by magnitude, zeroing the rest.
   (b) **Elect Sign:** At each parameter position, elect the dominant sign by majority magnitude vote.
   (c) **Disjoint Merge:** Average only the sign-consistent values at each position.

   This yields $\tilde{A}_{\text{merged}} \in \mathbb{R}^{r \times d_{\text{in}}}$.
4. **Merge $\tilde{B}$ via averaging:** Compute $\tilde{B}_{\text{merged}} = \frac{1}{N} \sum_{i=1}^{N} \tilde{B}_i$. After alignment, each $\tilde{B}_i = B_i Q_i^T$ expresses its output projection relative to the *same* rotated r-dimensional input-side coordinate system established by GPA. In a well-trained LoRA adapter, B_i captures the output directions most activated by the task-specific features that A_i extracts; once the A_i are aligned, the $\tilde{B}_i$ are all responding to the same input directions, making their average a meaningful ensemble rather than a combination of incommensurable maps. We acknowledge that this reasoning assumes the tasks share sufficient output structure that averaging is not destructive; when tasks are highly dissimilar, averaging $\tilde{B}$ may wash out task-specific output directions. This is precisely what the TIES-on-$\tilde{B}$ ablation (Section 4) is designed to test. As a **ablation**, we also evaluate applying TIES to the $\tilde{B}_i$ matrices.
5. **Reconstruct:** Compute the merged task update: $\Delta W_{\text{merged}} = \tilde{B}_{\text{merged}} \cdot \tilde{A}_{\text{merged}}$.
6. **Apply:** Add the merged delta to the pretrained weights: $W_{\text{merged}} = W_{\text{pre}} + \lambda \cdot \Delta W_{\text{merged}}$, where λ is a scaling coefficient tuned on held-out validation data.

Critically, because TIES-Merging is applied to $\tilde{A}_i$ *before* reconstruction, the orthogonal rotations Q_i are not cancelled out. The nonlinear conflict resolution (trimming, sign election, masking) operates in the aligned coordinate system, where element-wise comparisons across adapters are geometrically meaningful. Only after merging is the result multiplied by $\tilde{B}_{\text{merged}}$ to produce the final full-rank update.

Note that this pipeline operates entirely on matrices of size $r \times d_{\text{in}}$ (for $\tilde{A}$) and $d_{\text{out}} \times r$ (for $\tilde{B}$), never materialising full $d_{\text{out}} \times d_{\text{in}}$ matrices until the final reconstruction of a *single* merged update. This preserves the alignment-phase memory efficiency that motivates our approach.

KnOTS observes that alignment is only beneficial when combined with nonlinear conflict-resolution methods like TIES or DARE; applying Task Arithmetic (simple weighted averaging) to the full-rank updates in the aligned space yields no improvement, because the linear combination commutes with the orthogonal rotations. We expect this finding to hold in our factor-space setting as well, noting that applying Task Arithmetic separately to the factors is not mathematically equivalent to applying it to the full-rank updates due to cross-term interactions in the product $\tilde{B}_{\text{merged}} \cdot \tilde{A}_{\text{merged}}$. We will verify this experimentally.

1.6.4 Relationship to KnOTS

Our GPA approach and KnOTS solve related but structurally different problems. KnOTS decomposes the *concatenated full-rank* task updates $[\Delta W^{(1)}; \ldots; \Delta W^{(n)}]$ (since KnOTS operates on the materialised full-rank updates $\Delta W^{(i)} = B_i A_i$) via a single SVD to obtain a shared basis $U\Sigma$ and task-specific components $[V^{(i)}]^T$, then merges the $V^{(i)}$ while holding $U\Sigma$ fixed. Our method

computes iterative rotations of the *low-rank factors* A_i directly via GPA, producing aligned factors $\tilde{A}_i$ (input-side, merged via TIES) and $\tilde{B}_i$ (output-side, averaged), without ever materialising the full ΔW_i matrices during alignment or merging.

The structural correspondence is:

KnOTS	**GPA (Ours)**	**Role**
$U\Sigma$ (shared by construction)	$\tilde{B}_i$ (averaged into shared)	Output-side projection
$[V^{(i)}]^T$ (merged)	$\tilde{A}_i$ (merged via TIES)	Input-side projection

The key difference is computational: for Qwen2.5-1.5B with hidden size 1536 and $r = 16$, KnOTS requires constructing and decomposing a concatenated matrix of size $1536 \times (N \times 1536)$, whereas our GPA operates on $r \times d_{\text{in}}$ matrices (16×1536) and performs SVDs on $r \times r$ matrices (16×16). Each full ΔW occupies $1536 \times 1536 \times 4 = 9.4$ MB versus $16 \times 1536 \times 4 = 96$ KB for each A_i: a **98× memory reduction per matrix**. This is the source of the $\mathcal{O}(1)$ alignment-phase scaling advantage.

1.6.5 The LR-KnOTS Baseline and Why GPA is Different

A natural question is whether the alignment-phase memory advantage requires the full GPA machinery, or whether a simpler one-shot approach suffices. We define a critical baseline, **LR-KnOTS** (Low-Rank KnOTS), which applies the KnOTS strategy directly to the A factors: concatenate $[A_1; A_2; \ldots; A_N] \in \mathbb{R}^{Nr \times d_{\text{in}}}$ column-wise and compute a single SVD, decomposing into $U_A \Sigma_A [V_A^{(1)}, \ldots, V_A^{(N)}]^T$, then merge the $V_A^{(i)}$ via TIES and reconstruct as in KnOTS. This baseline has an identical alignment-phase memory footprint to our GPA method and is a natural low-rank adaptation of KnOTS.

GPA differs from LR-KnOTS in two important respects. First, LR-KnOTS applies a single global SVD to the stacked A factors, finding a shared left singular space U_A that is optimal in a least-squares sense over the concatenation. This means the consensus is implicitly weighted by the Frobenius norm of each A_i: adapters trained on larger or harder tasks, which tend to develop larger-norm factors, will dominate the shared basis. By contrast, GPA iteratively computes the consensus as a simple mean of the rotated factors $Q_i A_i$, so each adapter contributes equally regardless of its norm. This equal-contribution property is more appropriate when adapters are trained on tasks of unequal difficulty or dataset size, which is typical in GLUE. Second, GPA produces explicit per-model rotation matrices Q_i that can be inspected and compared as a measure of how far each model's subspace deviated from the consensus, providing interpretable diagnostics that LR-KnOTS does not.

LR-KnOTS is included as a **primary baseline** in Section 4. If GPA outperforms LR-KnOTS empirically, this constitutes direct evidence that iterative equal-weighting alignment provides a benefit beyond the memory savings alone. If the two perform comparably, the contribution of GPA shifts to its convergence guarantees, equal-contribution property, and interpretability.

2 Computational Analysis of GPA Overhead

A natural concern is whether the iterative GPA algorithm introduces significant I/O or communication overhead, particularly due to frequent transfers of model parameters between system memory and VRAM. This section demonstrates that the overhead is **negligible** and that the entire GPA computation should run on CPU.

2.1 The GPA Alternating Optimization

The standard GPA algorithm (Gower [2]) alternates two steps until convergence:

1. **Update rotations (fix C):** For each model i, solve the ordinary Procrustes problem via SVD of the $r \times r$ matrix $M_i = CA_i^T$. Decompose $M_i = U\Sigma V^T$ and set $Q_i = UV^T$.
2. **Update consensus (fix all Q_i):** Compute $C = \frac{1}{N}\sum_{i=1}^{N} Q_i A_i$.

Each step provably decreases the objective in Equation (1), guaranteeing monotonic convergence to a local minimum.

2.2 Per-Iteration Cost for $r = 16$, $d = 1536$, $N = 10$

The critical observation is that CA_i^T is an $r \times r$ matrix (e.g., 16×16), so each SVD operates on a tiny matrix. The per-iteration breakdown:

Operation	**Count**	**Cost Each**	**Total**
Matrix multiply $C \cdot A_i^T$: $(16 \times 1536) \times (1536 \times 16)$	10	$\sim$50 μs	$\sim$500 μs
SVD of 16×16 matrix	10	$\sim$30 μs	$\sim$300 μs
Matrix multiply $Q_i \cdot A_i$: $(16 \times 16) \times (16 \times 1536)$	10	$\sim$10 μs	$\sim$100 μs
Consensus mean computation	1	negligible	$\sim$10 μs
Total per iteration			**$\sim$1 ms**

Table 1: Per-iteration computational cost of GPA for one LoRA layer.

2.3 Memory Requirements

Each $r \times d$ matrix in float32 occupies 96 KB ($16 \times 1536 \times 4$ bytes). For $N = 10$, the input matrices, consensus, and rotation matrices together require approximately **1.1 MB** which is small enough to fit in CPU L3 cache. Even with all 28 layers of Qwen2.5-1.5B processed simultaneously, total memory is approximately 10 MB per adapter set. **No GPU involvement is needed or desirable.**

2.4 Why CPU, Not GPU

PyTorch benchmarks demonstrate that GPU SVD for small matrices can be 100–500$\times$ slower than CPU due to kernel launch overhead, memory allocation latency, and the inability of small operations to saturate GPU parallelism. The recommendation is to keep GPA entirely in NumPy/SciPy using LAPACK's divide-and-conquer SVD (`DGESDD`).

2.5 Convergence Rate

Ling [4] proved that the Generalized Power Method, equivalent to GPA's alternating minimization with spectral initialization, converges *linearly* to the global optimum when the signal-to-noise ratio is sufficiently large. In practice, GPA converges in 5–20 iterations for typical problems; for well-conditioned low-rank matrices like trained LoRA adapters from the same base model, we expect **3–10 iterations** at a tolerance of 10^{-6} relative change in the objective.

Total wall-clock time for GPA on one layer: $\sim$3–10 ms. For all 28 layers of Qwen2.5-1.5B, the entire alignment procedure takes roughly **80–280 ms** which is less time than a single forward pass. The computational bottleneck in the pipeline will be loading LoRA weights from disk, not the alignment computation itself.

3 Synthetic Validation Protocol

Before applying GPA to real LoRA adapters, we will validate the algorithm's convergence and robustness on synthetic data. Our protocol is adapted from Pizarro & Bartoli [5], who systematically varied noise level, number of configurations, and dimensionality across repeated random trials.

3.1 Experiment 1: Ground-Truth Rotation Recovery

Generate a ground-truth matrix $A^* \in \mathbb{R}^{16\times 1536}$ by sampling entries from $\mathcal{N}(0, 1/d)$. For each of N adapters, produce $A_i = Q_i A^* + \sigma E_i$, where Q_i is a random orthogonal matrix (from QR decomposition of a random Gaussian matrix) and E_i is i.i.d. Gaussian noise. Run GPA to recover rotations $\hat{Q}_i$. Measure:

- Rotation recovery error: $\frac{1}{N}\sum_i \|\hat{Q}_i - Q_i\|_F^2$ (after resolving global rotation ambiguity).
- Consensus relative error: $\|C - A^*\|_F / \|A^*\|_F$.
- Alignment residual: $\frac{1}{N}\sum_i \|\hat{Q}_i A_i - C\|_F^2$.

Vary $\sigma \in \{0, 0.01, 0.05, 0.1, 0.2, 0.5\}$, $N \in \{3, 5, 10\}$, and $r \in \{4, 8, 16, 32\}$, with 100 random trials per configuration.

3.2 Experiment 2: Convergence Curves

Fix $\sigma = 0.1$, $N = 10$, $r = 16$. Record the residual sum-of-squares at each GPA iteration. Plot residual vs. iteration number (expecting monotonic decrease on a log scale). Report iterations to convergence at tolerance 10^{-6} and wall-clock time.

3.3 Experiment 3: Robustness to Non-Orthogonal Perturbations

Real LoRA adapters are not exact rotations of a shared matrix. Test $A_i = (Q_i + \delta S_i)A^* + \sigma E_i$, where S_i are random matrices with unit Frobenius norm and $\delta \in \{0, 0.01, 0.05, 0.1, 0.2\}$ controls departure from orthogonality. This tests how gracefully GPA degrades when its assumptions are violated.

3.4 Experiment 4: Structured LoRA-Like Ground Truth

Generate A^* with geometrically decaying singular values (mimicking trained LoRA matrices). Add task-specific rank-1 perturbations P_i (simulating task-specific directions). Test whether GPA can separate the shared component from task-specific components. Measure subspace overlap via principal angles between $\text{span}(C)$ and $\text{span}(A^*)$.

This entire synthetic suite can be implemented in pure NumPy in approximately one day and should precede any experiments on real models.

4 Requirements Specification

4.1 Main Objectives

- **High Priority: Replicate Core Phenomena & Baselines**
  - Deploy Qwen2.5-1.5B utilizing 4-bit NF4 quantization to fit within the 8 GB VRAM limit (∼0.77 GB for model weights, leaving ∼5–6 GB headroom).
  - Train 4–5 task-specific QLoRA adapters on a GLUE subset (SST-2, MNLI, QNLI, CoLA, RTE).
  - Replicate standard merging baselines (Task Arithmetic, TIES, DARE) on these adapters.
- **High Priority: Synthetic Validation of GPA**
  - Implement and run the four-experiment synthetic validation protocol (Section 3).
  - Demonstrate convergence in ≤ 10 iterations and rotation recovery under moderate noise.
- **Medium Priority: Implement and Evaluate GPA-Aligned Merging**
  - Develop the iterative SVD algorithm to compute optimal Q_i and consensus C.
  - Implement the LR-KnOTS baseline (Section 1.6.5): column-wise concatenation of A factors followed by a single SVD and TIES merge of the resulting $V^{(i)}$ matrices.
  - Implement the full GPA merging pipeline: GPA alignment → TIES on aligned $\tilde{A}$ factors with $\tilde{B}$ averaging (Section 1.6).
  - Evaluate merged models on all tasks. Primary comparisons: GPA+TIES vs. LR-KnOTS+TIES vs. TIES alone vs. Task Arithmetic vs. simple averaging.
- **Low Priority: Analysis and Ablations**
  - Analyse CKA (Centered Kernel Alignment) before and after GPA alignment.
  - Vary number of merged adapters (2, 3, 4, 5) and LoRA rank (8, 16, 32).
  - Ablation: consensus replacement vs. TIES in aligned factor space.
  - Ablation: TIES on $\tilde{B}$ factors vs. simple averaging of $\tilde{B}$ factors.

4.2 Success Criteria and Negative Result Protocol

The **primary hypothesis** is that GPA+TIES outperforms TIES on unaligned factors by a statistically meaningful margin ($\geq$1% average accuracy across GLUE tasks, consistent across at least 3 of 5 tasks). The **secondary hypothesis** is that GPA+TIES outperforms LR-KnOTS+TIES, demonstrating that iterative equal-weighting alignment provides a benefit beyond the memory savings alone.

These hypotheses admit four outcomes, each of which yields a coherent dissertation:

1. *Both hypotheses confirmed:* GPA is validated as a stronger alignment method than both no-alignment and single-pass low-rank concatenation.
2. *Primary confirmed, secondary not:* GPA and LR-KnOTS perform comparably; the contribution shifts to GPA's convergence guarantees, equal-contribution property, and the interpretable Q_i rotation matrices.
3. *Primary confirmed only weakly:* Improvement is present but small or inconsistent across tasks; analysis focuses on understanding when alignment helps via CKA and alignment residuals.
4. *Neither confirmed:* The dissertation documents this negative result and uses the synthetic experiments (particularly Experiment 3) to investigate whether GPA's orthogonality assumptions are violated by QLoRA adapters trained on GLUE, providing actionable guidance for future work.

4.3 Deliverables

- A highly memory-efficient Python codebase for multi-way iterative Procrustes alignment, including the LR-KnOTS baseline implementation.
- Master's Dissertation documenting the algorithm, synthetic validation, merging experiments, and analysis (Target: 1st May 2026).

5 Project Plan

Task	**Start**	**End**	**Hours (Approx.)**
Infrastructure & smoke test: environment setup, quantization verification, single-task (SST-2) adapter training to validate the full pipeline. Contingency: if bitsandbytes or PEFT issues arise, catch and resolve before committing to full training.	Week 1, Days 1–2	Week 1, Days 1–2	15
Remaining QLoRA adapter training (MNLI, QNLI, CoLA, RTE); can run overnight on cluster. Contingency: if training exceeds this window, reduce to 3 tasks (SST-2, MNLI, QNLI) as a valid $N = 3$ proof-of-concept.	Week 1, Days 3–5	Week 1, Days 3–5	30
Synthetic GPA validation (Experiments 1–4) & baseline merging implementation (TIES, DARE, TA, LR-KnOTS)	Week 2	Week 2	50
GPA alignment implementation (NumPy/SciPy on CPU) & factor-space merging pipeline (TIES on $\tilde{A}$, averaging $\tilde{B}$). Begin dissertation write-up: introduction and related work.	Week 3	Week 3	50
Full experiment matrix: evaluate all merged models on all tasks. Continue dissertation write-up: methods section.	Week 4	Week 4	45
Ablations (rank, N, consensus replacement, TIES vs. averaging on $\tilde{B}$), CKA analysis, figure generation. Continue dissertation write-up: experiments and results.	Week 5	Week 5	45
Dissertation synthesis, revision, and final visualizations.	Week 6	Week 6	50

Table 2: Revised 6-week project plan. Adapter training is split across Days 1–2 (smoke test) and Days 3–5 (full training) to surface infrastructure issues early. Dissertation writing begins in Week 3 and runs in parallel with experiments.

6 Resources

6.1 Literature (Research Papers)

This research will synthesize foundational literature on model merging, SVD alignment, and parameter-efficient fine-tuning:

6.1.1 SVD and Subspace Alignment for Merging

- Stoica, G. et al. (2025). "Model merging with SVD to tie the knots." ICLR. [6]

6.1.2 Standard Merging Baselines

- Ilharco, G. et al. (2023). "Editing models with task arithmetic." ICLR. [3]
- Yadav, P. et al. (2023). "TIES-Merging: Resolving interference when merging models." NeurIPS. [7]
- Yu, L. et al. (2024). "Language models are super mario: Absorbing abilities from homologous models as a free lunch." (DARE) ICML. [8]

6.1.3 Procrustes Analysis and Geometry

- Gower, J. C. (1975). "Generalized procrustes analysis." Psychometrika. [2]
- Achara, A. et al. (2026). "Multi-Way Representation Alignment." arXiv preprint arXiv:2602.06205. [1]
- Pizarro, D. & Bartoli, A. (2011). "Global optimization for optimal generalized procrustes analysis" CVPR. [5]
- Ling, S. (2023). "Near-optimal bounds for generalized orthogonal Procrustes problem via generalized power method." Applied and Computational Harmonic Analysis, 66:62–100. [4]

6.2 Computational Resources

- **GPU Cluster Node (weatherwax):** The project will strictly utilize the available weatherwax node. This node contains 6× NVIDIA GeForce RTX 2080 GPUs strictly limited to 8.00 GB of Video RAM.
- **GPA on CPU:** Because the GPA algorithm operates exclusively on the $r \times d$ LoRA matrices (96 KB each at $r = 16$, $d = 1536$), all SVD operations run on CPU via NumPy/SciPy in single-digit milliseconds per layer (see Section 2). No VRAM is consumed by the alignment procedure.
- **Base Model:** Qwen2.5-1.5B (1.54B parameters, hidden size 1536, 28 layers) in 4-bit NF4 quantization (∼0.77 GB). Each QLoRA adapter at rank 16 adds ∼10M trainable parameters (∼40 MB).
- **Software:** Python, NumPy/SciPy (for GPA), PyTorch, bitsandbytes (for 4-bit NF4 quantization), and the HuggingFace `peft` library.

References

[1] Akshit Achara, Tatiana Gaintseva, Mateo Mahaut, Pritish Chakraborty, Viktor Stenby Johansson, Melih Barsbey, Emanuele Rodolà, and Donato Crisostomi. Multi-way representation alignment. *arXiv preprint arXiv:2602.06205*, 2026.

[2] John C Gower. Generalized procrustes analysis. *Psychometrika*, 40(1):33–51, 1975.

[3] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

[4] Shuyang Ling. Near-optimal bounds for generalized orthogonal procrustes problem via generalized power method. *Applied and Computational Harmonic Analysis*, 66:62–100, 2023.

[5] Daniel Pizarro and Adrien Bartoli. Global optimization for optimal generalized procrustes analysis. In *CVPR 2011*, pages 2409–2415. IEEE, 2011.

[6] George Stoica, Pratik Ramesh, Boglarka Ecsedi, Leshem Choshen, and Judy Hoffman. Model merging with svd to tie the knots. *arXiv preprint arXiv:2410.19735*, 2024.

[7] Prateek Yadav, Derek Tam, Leshem Choshen, Colin A Raffel, and Mohit Bansal. Ties-merging: Resolving interference when merging models. *Advances in neural information processing systems*, 36:7093–7115, 2023.

[8] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Forty-first International Conference on Machine Learning*, 2024.